# Lecture 07 — Introduction to Arrays

DSA Mastery Series | C++ Focused | Arrays, Strings & Two Pointers Track

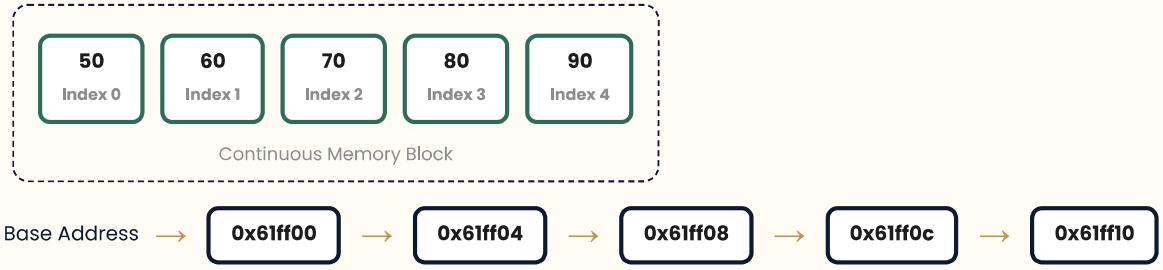
## What is an Array?

CORE INTUITION

**Array ek Similar Data Type ka Collection hota hai jo Continuous Memory Locations me store hota hai.**

Socho ek school me 100 students hain aur har student ka marks store karna hai. Agar 100 alag-alag variables banayein (`marks1, marks2, ... marks100`) to code manage karna impossible ho jayega. Array isi problem ko solve karta hai — ek hi naam se multiple values ko ek jagah store kar leta hai.

**Memory Visualization — Array of 5 Integers**




Har integer 4 bytes leta hai → address +4 se badhta hai

## Why Array?

THE PROBLEM

Agar 5 students ke marks store karne ho to 5 variables chahiye:

```
int marks1 = 50;
int marks2 = 60;
int marks3 = 70;
int marks4 = 80;
int marks5 = 90;
```

CRITICAL INSIGHT

Agar **100 students** ke marks store karne ho to 100 variables banana practical nahi hai. Code bulky, unreadable aur unmaintainable ho jayega. Isliye **Array** ka use karte hain — ek naam, multiple values.

# Array Declaration

**SYNTAX**

`data_type array_name[size];`

Size compile time pe fix ho jata hai. Ye batata hai ki array me kitne elements store ho sakte hain.

```
// Size = 5 → Index: 0, 1, 2, 3, 4
int marks[5];

// Memory me 5 contiguous blocks reserve ho gaye
// Output visual: [ _, _, _, _, _ ]
```

**CAUTION**

C++ me **array ka size compile-time constant** hona chahiye (static array ke case me). Agar variable size chahiye to `vector` use karo ya dynamic allocation (`new int[n]`) karo.

# Array Initialization

## Method 1: Index-wise (One by One)

```
marks[0] = 50;      // [50,  _,  _,  _,  _]
marks[1] = 60;      // [50, 60,  _,  _,  _]
marks[2] = 70;      // [50, 60, 70,  _,  _]
marks[3] = 80;      // [50, 60, 70, 80,  _]
marks[4] = 90;      // [50, 60, 70, 80, 90]
```

## Method 2: Direct Initialization

```
int mark[5] = {10, 20, 30, 40, 50};
// Output: [10, 20, 30, 40, 50]
```

## Method 3: Compiler Auto-Size

```
int arr1[] = {20, 30, 40, 50, 60};
// Compiler automatically size = 5 decide kar lega
```

**BEST PRACTICE**

Agar elements pehle se pata hain to **direct initialization** prefer karo. Agar runtime pe input lena hai to **index-wise** ya **loop se input** lo.

# 0-Based Indexing

**KEY INSIGHT**

Array me **pehla element Index 0** pe hota hai aur **last element Index (n-1)** pe. Ye 0-based indexing kehlaata hai.

| Property | Formula |
|---|---|
| Array Size | `n` |
| First Index | `0` |
| Last Index | `n - 1` |
| Total Elements | `n` |

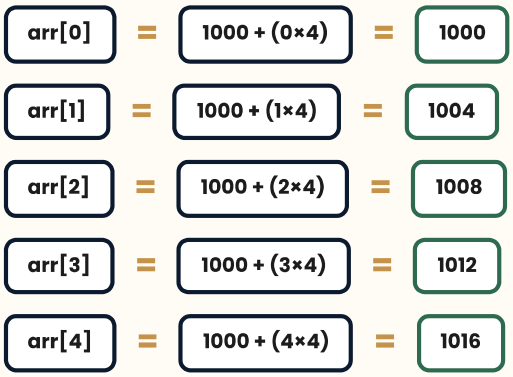
### Why 0-Based Indexing?

MEMORY ADDRESS FORMULA

**Address of arr[i] = Base Address + (i × Size of Data Type)**

Agar 1-based indexing hoti to formula me `(i - 1)` extra calculation karni padti — ek subtraction har access pe. 0-based indexing se CPU directly offset calculate kar sakta hai bina kisi extra operation ke. Ye **performance optimization** hai.

**Address Calculation — int arr[5] at Base = 1000**




## Traversing an Array

ALGORITHM

Loop chalao `i = 0` se `i < n` tak. Har iteration me `arr[i]` ko access karo. Ye **O(n)** time me complete hota hai.

```
int mark[5] = {10, 20, 30, 40, 50};

cout << "Array Elements: ";

for (int i = 0; i < 5; i++) {
    cout << mark[i] << " ";
}

// Output: Array Elements: 10 20 30 40 50
```

EDGE CASE / PITFALL

Loop condition me `hamesha i < n rakho`, `i <= n` mat rakho — ye **out-of-bounds** access hoga aur **undefined behavior** milega ya program crash ho jayega.

## User Input in Array

TWO-STEP PROCESS

**Step 1:** User se size `n` lo.

**Step 2:** Loop chalake `n` elements input lo, phir `n` elements output karo.

```cpp
int n;
cout << "Tell me the Size of the Array: ";
cin >> n;

int arr[n];     // VLA (Variable Length Array) — GCC supports, not standard C++

cout << "Enter " << n << " Elements:" << endl;

// Input
for (int i = 0; i < n; i++) {
    cin >> arr[i];
}

// Output
cout << "Array Elements: ";
for (int i = 0; i < n; i++) {
    cout << arr[i] << " ";
}
```

WARNING

`int arr[n]` jaha `n` runtime variable hai — ye **VLA (Variable Length Array)** hai. GCC me chalta hai par **standard C++ me allowed nahi hai**. Production code me `vector<int> arr(n)` use karo.

◈ ◈ ◈

## Problem 1 — Find Highest Element

PROBLEM STATEMENT

**Input:** `[70, 30, 40, 30, 20]`

**Output:** `Highest Element : 70`

### Core Intuition

INTUITION / INSIGHT

Pehle element ko "**sabse bada**" maan lo. Phir pure array me traverse karo — agar koi element current highest se bada mile to usse highest bana do. Last tak pahunchne pe jo bachega wahi answer hai.

### Approach — Linear Scan (Optimal)

BEST PRACTICE / OPTIMIZATION

Single pass me kaam ho jata hai. `No extra space` chahiye. Ye **Kadane-like greedy thinking** hai — har step pe best answer maintain karo.

```cpp
// Find Highest Element in Array
int main() {
    int arr[] = {70, 30, 40, 30, 20};

    int highest = arr[0];   // first element ko highest maan liya

    for (int i = 0; i < 5; i++) {
        if (arr[i] > highest) {
            highest = arr[i];   // bada mila to update
        }
    }

    cout << "Highest Element: " << highest;
    return 0;
}
```

## Step-by-Step Dry Run

DRY RUN / VISUALIZATION

**Array:** `[70, 30, 40, 30, 20]` | **Initial highest = 70**

| i | arr[i] | arr[i] > highest? | Action | highest |
|---|---|---|---|---|
| 0 | 70 | 70 > 70 → No | Skip | 70 |
| 1 | 30 | 30 > 70 → No | Skip | 70 |
| 2 | 40 | 40 > 70 → No | Skip | 70 |
| 3 | 30 | 30 > 70 → No | Skip | 70 |
| 4 | 20 | 20 > 70 → No | Skip | 70 |

✓ Final Answer: 70

COMPLEXITY ANALYSIS

| Metric | Value | Reason |
|---|---|---|
| Time Complexity | O(n) | Har element ek baar check hota hai |
| Space Complexity | O(1) | Extra variable sirf `highest` |

# Problem 2 — Linear Search

PROBLEM STATEMENT

Array me ek specific element search karna hai. Agar mil jaye to uska **index** print karo, nahi to **"not present"** message do.

**Example 1:** Array = [10,20,30,40,50], Search = 30 → Index: 2
**Example 2:** Array = [10,70,56,80,90], Search = 40 → Element is not present

## Core Intuition

INTUITION / INSIGHT

Linear Search me hum array ke **har element ko ek-ek karke check** karte hain jab tak element mil na jaye ya pura array khatam na ho jaye. Jaise ek drawer me cheezein dhoondh rahe ho — ek-ek karke har drawer kholte jao.

## Approach — Sequential Scan

ALGORITHM

1. `found = false` flag set karo.
2. Loop se har `arr[i]` ko `search` se compare karo.
3. Match milte hi `found = true` karo, result print karo, aur `break` karo.
4. Loop ke baad agar `found == false` hai to "not present" print karo.

```cpp
// Linear Search in Array
int main() {
    int arr[] = {10, 70, 56, 80, 90};
    int search = 40;
    bool found = false;

    for (int i = 0; i < 5; i++) {
        if (arr[i] == search) {
            cout << "Search Element: " << arr[i] << endl;
            cout << "Index: " << i << endl;
            found = true;
            break;   // mil gaya, aage check karne ki zaroorat nahi
        }
    }

    if (!found) {
        cout << "Element is not present in the Array";
    }
    return 0;
}
```

## Step-by-Step Dry Run

DRY RUN / VISUALIZATION

**Array:** `[10, 70, 56, 80, 90]` | **Search:** `40`

| i | arr[i] | arr[i] == 40? | Action |
|---|---|---|---|
| 0 | 10 | 10 == 40 → No | Continue |
| 1 | 70 | 70 == 40 → No | Continue |
| 2 | 56 | 56 == 40 → No | Continue |
| 3 | 80 | 80 == 40 → No | Continue |
| 4 | 90 | 90 == 40 → No | Continue |

✗ Loop khatam → found = false → "Element is not present in the Array"

COMPLEXITY ANALYSIS

| Case | Time Complexity | Explanation |
|---|---|---|
| Best Case | O(1) | Element pehle position pe mil jaye |
| Worst Case | O(n) | Element last pe ho ya present hi na ho |
| Average Case | O(n) | Random position pe element |
| Space | O(1) | Extra space nahi lagti |

INTERVIEW TIP

Interviewer pooch sakta hai: *"Sorted array me search karna ho to kya use karoge?"* → Answer: **Binary Search (O(log n))**. Linear Search sirf **unsorted data** ke liye suitable hai.

## Edge Cases & Gotchas

EDGE CASE / PITFALL

- **Empty Array:** `n = 0` ho to loop execute hi nahi hoga. Highest/Search ka logic pehle se handle kare.
- **Single Element:** `arr[0]` hi answer hoga — logic automatically handle karta hai.
- **All Same Elements:** Highest = koi bhi element. Search = pehla match return hoga (because of `break`).
- **Negative Numbers:** `INT_MIN` se initialize karna safe hota hai agar `arr[0]` na maan sakein.
- **Out of Bounds:** `arr[n]` access karna → undefined behavior. Hamesha `i < n` rakho.

## Pattern Transfer / Related Problems

PATTERN TRANSFER

| Problem | Approach | Complexity |
|---|---|---|
| Find Minimum Element | Same as Highest, < comparison | O(n) |
| Find Second Highest | Two variables track karo | O(n) |
| Count Occurrences | Counter variable, no `break` | O(n) |
| Find All Indices | Vector me indices store karo | O(n) |
| Binary Search | Sorted array pe mid compare | O(log n) |

## Key Takeaways

TAKEAWAY / SUMMARY

- Array **same data type** ke elements ka **continuous memory** collection hai.
- Indexing **0-based** hoti hai: First = 0, Last = n-1.
- Memory address formula: `Base + (i × sizeof(type))`
- Traversal hamesha `for (i=0; i<n; i++)` se karo.
- Highest/Minimum — **single pass greedy** approach, O(n) time, O(1) space.
- Linear Search — **unsorted data** ke liye, O(n) worst case.
- `break` se early exit possible jab element mil jaye.

### ◆ QUICK REVISION CHEATSHEET ◆

**Array Definition**
Same data type ka collection, continuous memory me store hota hai.

**Declaration**
`int arr[5];` → 5 integers reserve

**Initialization**
Index-wise: `arr[0]=10;`
Direct: `int arr[]={10,20,30};`

**0-Based Indexing**
First = 0, Last = n-1
Address = Base + (i × sizeof(type))

**Traversal**
```
for(int i=0; i<n; i++)
cout << arr[i];
```

**Input/Output**
```
cin >> arr[i];
cout << arr[i];
```

**Find Highest**
```
highest = arr[0];
if(arr[i] > highest) update;
```

**Linear Search**
`bool found=false;`
Match milte hi `break;`
TC: O(n)  SC: O(1)

**Common Mistakes**
- `i <= n` → Out of Bounds
- VLA `int arr[n]` → Not standard C++
- Highest = 0 se init → fails on negatives

**Interview Qs**
- Sorted array me search? → Binary Search
- Duplicate elements? → Count karo, vector store karo
- 2D array? → Row-major order me store hota hai